

Risposte Discorsive – Sistemi Intelligenti

Come rispondere all'orale e alle domande aperte

A.A. 2025–2026

Indice

1 Test di Turing e IA Forte vs Debole

Domanda: Cos'è il Test di Turing? Cosa dice esattamente?

Il Test di Turing, proposto da Alan Turing nel 1950 nel suo articolo “Computing Machinery and Intelligence”, è un criterio operativo per stabilire se una macchina può essere considerata intelligente. L'idea è la seguente: un giudice umano interagisce via tastiera con due interlocutori che non può vedere direttamente, uno dei quali è un essere umano e l'altro è una macchina. Se il giudice non riesce a distinguere l'uno dall'altro dopo una conversazione libera, allora la macchina supera il test e può essere considerata intelligente.

La cosa importante da sottolineare è che il test non richiede che la macchina risponda *correttamente* alle domande, né che le *comprenda* davvero: richiede che **imiti il comportamento umano** nel rispondere. Questo è un punto critico, perché un essere umano può sbagliare, può essere distratto, può rispondere in modo incoerente, e la macchina deve essere in grado di fare altrettanto in modo convincente.

Le capacità che un sistema dovrebbe avere per superare il test riguardano l'elaborazione del linguaggio naturale, la rappresentazione della conoscenza, il ragionamento automatico e l'apprendimento. John Searle ha criticato il test con l'argomento della Stanza Cinese: una persona che segue regole per manipolare simboli cinesi senza capirne il significato starebbe “superando” il test senza alcuna vera comprensione. Questo ha aperto il dibattito tra IA forte e IA debole.

Domanda: Qual è la differenza tra IA Forte e IA Debole?

Quando parliamo di IA debole intendiamo sistemi che *simulano* comportamenti intelligenti per uno scopo specifico, senza possedere comprensione reale o stati mentali propri. Un motore di scacchi, un sistema di riconoscimento vocale, un chatbot: questi sono tutti esempi di IA debole perché risolvono problemi definiti, ma non “capiscono” quello che stanno facendo nel senso in cui lo fa un essere umano.

L'IA forte, al contrario, ipotizza che una macchina possa possedere vera intelligenza, coscienza e stati mentali equivalenti a quelli umani. Non si tratta di simulare, ma di *essere* intelligente. È ancora un obiettivo teorico e filosofico, non raggiunto. Il libro di testo adotta l'approccio “**agire razionalmente**”: costruire agenti che massimizzano la misura di prestazione attesa, indipendentemente dal fatto che lo facciano “come un umano”.

2 Agenti e Ricerca

Domanda: Cos'è un agente razionale? Quali sono le componenti di un problema di ricerca?

Un agente è qualsiasi entità capace di percepire l'ambiente attraverso sensori e di agire su di esso tramite attuatori. Un agente è *razionale* quando sceglie l'azione che massimizza le sue prestazioni attese sulla base della sequenza di percezioni ricevute, della conoscenza disponibile e delle azioni possibili. È importante distinguere razionalità da onniscienza: un agente razionale fa del meglio che può con le informazioni che ha, ma non è infallibile perché il mondo contiene sempre fattori ignoti.

Quando formuliamo un problema di ricerca dobbiamo definire cinque elementi fondamentali. Lo **stato iniziale** è la configurazione di partenza dell'agente nel mondo. La **funzione successore** (o modello di transizione) descrive gli effetti di ogni azione: dato uno stato s e un'azione a , restituisce il nuovo stato risultante. Il **test obiettivo** determina se uno stato è una soluzione. Il **costo del cammino** è la somma dei costi di ogni passo compiuto. Infine, una **soluzione** è la sequenza di azioni che porta dallo stato iniziale a uno stato obiettivo.

Questi elementi non vanno confusi con un CSP: in un problema di ricerca cerchiamo una *sequenza di azioni*, mentre in un CSP cerchiamo un *assegnamento* che soddisfi dei vincoli, senza la nozione di cammino o costo.

3 Algoritmi di Ricerca

Domanda: Cosa differenzia un algoritmo informato da uno non informato?

La differenza fondamentale è l'utilizzo di una **funzione euristica** $h(n)$. Un algoritmo non informato come BFS, DFS o UCS non sa nulla sulla posizione del goal rispetto allo stato corrente: esplora il grafo in modo cieco, seguendo solo la struttura dello spazio degli stati. Un algoritmo informato, come A* o la ricerca greedy, dispone invece di una stima $h(n)$ del costo per raggiungere il goal da n e la usa per guidare l'esplorazione verso le zone più promettenti.

È importante chiarire che la funzione costo $g(n)$ – il costo del cammino percorso finora – è usata anche dagli algoritmi non informati come UCS. Quello che distingue gli algoritmi informati non è quindi il costo, ma proprio la stima euristica del costo futuro.

Domanda: Descrivi BFS, DFS, IDDFS e UCS: completezza, ottimalità, complessità

Il **BFS** (Breadth-First Search) esplora il grafo per livelli: espande prima tutti i nodi a profondità d , poi quelli a profondità $d + 1$, e così via. Usa una coda FIFO come frontiera. È **completo** – trova sempre una soluzione se esiste – e **ottimale** quando

i costi degli archi sono tutti uguali, perché il primo cammino trovato è anche il più corto in numero di passi. Il difetto principale è la **complessità spaziale**: mantiene in memoria tutti i nodi del livello corrente, il che con fattore di ramificazione b e soluzione a profondità d richiede $O(b^d)$ nodi. Questo lo rende impraticabile per problemi con grandi spazi degli stati.

Il **DFS** (Depth-First Search) esplora in profondità lungo un ramo fino alla foglia, poi torna indietro e prova il successivo. Usa uno stack. Ha complessità spaziale $O(bm)$, dove m è la profondità massima – molto più efficiente del BFS. Tuttavia **non è completo** su grafi con cicli (può girare indefinitamente) e **non è ottimale**: può trovare soluzioni molto profonde prima di scoprirne di più vicine.

L'**IDDFS** (Iterative Deepening DFS) combina i vantaggi di BFS e DFS nel modo più elegante possibile: esegue ripetutamente una ricerca DFS limitata a profondità crescente $l = 0, 1, 2, \dots$ fino a trovare la soluzione. Il vantaggio è che ha la **completezza e l'ottimalità di BFS** (con costi uniformi) ma la **complessità spaziale di DFS**: $O(bd)$. Il costo di riesaminare i nodi già visitati nelle iterazioni precedenti è sorprendentemente basso – asintoticamente non cambia la complessità temporale $O(b^d)$ – ed è di gran lunga preferibile al BFS per grandi spazi degli stati.

L'**UCS** (Uniform Cost Search) è BFS con priorità: la frontiera è una coda a priorità ordinata per $g(n)$, il costo del cammino percorso. Estrae sempre il nodo di costo minimo. È **completo e ottimale** anche con archi a costo variabile, purché i costi siano non negativi. Quando tutti i costi sono uguali a 1, degenera esattamente in BFS.

Algoritmo	Completo	Ottimale	Tempo	Spazio
BFS	Sì	Sì (costi uguali)	$O(b^d)$	$O(b^d)$
DFS	No	No	$O(b^m)$	$O(bm)$
IDDFS	Sì	Sì (costi uguali)	$O(b^d)$	$O(bd)$
UCS	Sì	Sì	$O(b^{C^*/\epsilon})$	$O(b^{C^*/\epsilon})$
A*	Sì	Sì (euristica ammiss.)	$O(b^d)$	$O(b^d)$

Domanda: Come funziona A*? Perché è ottimale?

A* è un algoritmo di ricerca informata che ordina la frontiera in base alla funzione $f(n) = g(n) + h(n)$, dove $g(n)$ è il costo del cammino già percorso da radice a n e $h(n)$ è la stima euristica del costo rimanente per raggiungere il goal. A ogni passo espande il nodo con f minimo.

L'ottimalità di A* dipende dalla proprietà dell'euristica. Su **alberi di ricerca**, basta l'ammissibilità: $h(n) \leq h^*(n)$. Su **grafi con lista chiusa**, dove i nodi espansi non vengono rivisitati, serve la **consistenza**: $h(n) \leq c(n, n') + h(n')$ per ogni arco. La consistenza garantisce che f non diminuisca mai lungo un cammino, ovvero che il primo percorso con cui si espande un nodo sia già quello ottimale.

La differenza tra A^* e UCS è proprio questa: UCS usa solo $g(n)$ e non sfrutta nessuna conoscenza del goal, mentre A^* usa anche $h(n)$ per orientare la ricerca. Con $h = 0$, A^* degenera in UCS.

Domanda: Cos'è RBFS e quando si usa?

RBFS (Recursive Best-First Search) è una versione di A^* progettata per operare con **memoria lineare**: mantiene in memoria solo il cammino corrente dalla radice al nodo attivo e i valori f dei nodi fratelli non ancora esplorati. Quando un ramo supera una soglia f_{limit} , l'algoritmo torna indietro, aggiorna il valore del nodo con il miglior f trovato nel sottoalbero esplorato, e poi ricomincia dal ramo con f minore.

Rispetto ad A^* , RBFS può riesaminare nodi più volte a causa dei ritorni all'indietro, aumentando il costo temporale; però la complessità spaziale è solo $O(bd)$ – analoga a IDDFS. È l'algoritmo da preferire quando la memoria disponibile è il fattore limitante.

Domanda: Cos'è un'euristica ammissibile? E una dominante?

Un'euristica $h(n)$ è **ammissibile** se non sovrastima mai il costo reale per raggiungere il goal: formalmente $h(n) \leq h^*(n)$ per ogni nodo n , dove $h^*(n)$ è il costo ottimo reale. Questo la rende “ottimistica”: preferisce sempre soluzioni buone. La proprietà fondamentale è che A^* con un'euristica ammissibile garantisce l'ottimalità su alberi di ricerca.

Se però lavoriamo su grafi con una lista chiusa – dove i nodi già espansi non vengono riesaminati – abbiamo bisogno di una condizione più forte: la **consistenza** (o monotonicità). Un'euristica è consistente se $h(n) \leq c(n, n') + h(n')$ per ogni arco (n, n') . La consistenza implica l'ammissibilità, ma non viceversa. Un'euristica consistente garantisce l'ottimalità di A^* anche su grafi.

Un'euristica h_1 **domina** h_2 se $h_1(n) \geq h_2(n)$ per ogni nodo n . Questo significa che h_1 è più informativa: fa stime più alte (senza mai superare il valore reale), e quindi A^* con h_1 espande meno nodi rispetto ad A^* con h_2 .

Una funzione costante zero, $h(n) = 0$, è tecnicamente ammissibile ma completamente non informativa: A^* con questa euristica degenera in UCS e non guida in alcun modo la ricerca verso il goal.

4 CSP – Problemi di Soddisfacimento di Vincoli

Domanda: Cos'è un CSP? Come si definisce una soluzione?

Un problema di soddisfacimento di vincoli (CSP) è una tripla $\langle X, D, C \rangle$ dove X è un insieme di variabili, D è l'insieme dei rispettivi domini e C è l'insieme dei vincoli. Ogni vincolo specifica quali combinazioni di valori sono ammissibili per un sottoinsieme di variabili.

Una soluzione di un CSP è un **assegnamento completo e consistente**: completo perché ogni variabile deve avere un valore, consistente perché nessun vincolo deve essere violato. Questo lo distingue nettamente dalla ricerca classica, dove la soluzione è una sequenza di azioni. Nel CSP non abbiamo né un cammino né un costo: vogliamo solo trovare una configurazione valida.

Domanda: Quali algoritmi si usano per risolvere un CSP? Qual è il più e il meno efficiente?

L'approccio più basilare – e meno efficiente – è il **generate and test**: si generano tutti i possibili assegnamenti completi dell'intero set di variabili, e per ciascuno si verifica se i vincoli sono soddisfatti. La complessità è $O(d^n)$ dove d è la dimensione del dominio e n è il numero di variabili, perché non sfrutta i vincoli durante la costruzione.

Il **backtracking** migliora notevolmente questo approccio: si assegnano le variabili una alla volta e, non appena un assegnamento parziale viola un vincolo, si torna indietro senza esplorare tutte le sue estensioni. Sfruttare la *commutatività* degli assegnamenti – ovvero il fatto che l'ordine in cui si assegnano le variabili non influenza il risultato – riduce ulteriormente l'ampiezza dell'albero.

L'algoritmo più efficiente tra quelli tipicamente discussi è il **back-jumping**. Mentre il backtracking cronologico, in caso di fallimento, torna sempre alla variabile assegnata immediatamente prima (anche se non è la responsabile del conflitto), il back-jumping identifica il **conflict set** – l'insieme delle variabili già assegnate che hanno causato il fallimento – e torna direttamente alla più recente tra queste. Questo evita di esplorare rami che non avrebbero comunque portato a una soluzione.

Domanda: Cos'è il back-jumping e il conflict set?

Quando una variabile X_k non può ricevere alcun valore senza violare un vincolo, il suo conflict set è l'insieme delle variabili già assegnate che sono in conflitto con X_k . Formalmente: $CS(X_k) = \{X_j : j < k, \exists \text{ vincolo tra } X_j \text{ e } X_k\}$.

Il back-jumping salta direttamente all'ultima variabile nel conflict set, bypassando tutte le variabili intermedie che non hanno contribuito al fallimento. Questo lo rende molto più efficiente del semplice backtracking cronologico, specialmente in problemi con molte variabili e vincoli distribuiti in modo sparso.

Domanda: Cos'è il Forward Checking nel CSP?

Il forward checking è una tecnica di **propagazione dei vincoli** che opera *durante* la costruzione dell'assegnamento: ogni volta che si assegna un valore alla variabile X_i , si controllano immediatamente tutti i vincoli che coinvolgono X_i e una variabile ancora non assegnata X_j , eliminando dal dominio di X_j i valori incompatibili con il valore appena scelto per X_i .

Il vantaggio è che questo rileva fallimenti molto prima del backtracking puro: se il dominio di una variabile futura diventa vuoto, si può già fare backtracking senza dover assegnare altre variabili. Tuttavia il forward checking non è completo nel

propagare le inconsistenze: non rileva le incompatibilità tra variabili future non ancora assegnate. Per questo, una tecnica più potente è l'**arc consistency**.

Domanda: Cos'è l'Arc Consistency (AC-3)?

Un arco (X_i, X_j) in un CSP è **arc-consistent** se per ogni valore nel dominio di X_i esiste almeno un valore nel dominio di X_j che è compatibile con il vincolo tra X_i e X_j . L'arc consistency si ottiene eliminando iterativamente i valori che violano questa condizione.

L'algoritmo **AC-3** gestisce una coda di archi da verificare. Prende un arco (X_i, X_j) , rimuove da D_i i valori per cui non esiste alcun supporto in D_j . Se D_i si riduce, aggiunge alla coda tutti gli archi (X_k, X_i) con $k \neq j$, perché la riduzione del dominio di X_i potrebbe rendere inconsistenti archi già controllati. L'algoritmo termina quando la coda è vuota o quando un dominio diventa vuoto (inconsistenza rilevata).

La differenza con il forward checking è che AC-3 propaga le riduzioni a cascata tra tutte le coppie di variabili, non solo tra la variabile appena assegnata e le future: è quindi molto più potente nel rilevare fallimenti anticipatamente, a costo di più computazione. L'arc consistency si può applicare sia come pre-processo prima del backtracking, sia interleaved durante la ricerca.

5 Ricerca Avversariale

Domanda: Come funziona l'algoritmo Minimax?

Minimax è un algoritmo per la ricerca in problemi con avversario, tipicamente giochi a somma zero a due giocatori con informazione perfetta come scacchi, dama o tris. Un gioco a **somma zero** è tale che il guadagno di un giocatore corrisponde esattamente alla perdita dell'altro: quello che guadagna MAX viene perso da MIN.

Il funzionamento di Minimax si basa sull'assunzione che entrambi i giocatori giochino in modo ottimale. L'algoritmo costruisce l'albero di gioco espandendo le mosse possibili a partire dallo stato corrente, alternativamente per MAX e per MIN. Una volta raggiunte le foglie – stati terminali o stati alla profondità limite – si applica la **funzione di utilità** (per stati terminali) o la **funzione di valutazione EVAL** (per stati non terminali alla profondità limite).

Il punto cruciale è che i valori vengono poi propagati **dalle foglie verso la radice**: i nodi MAX scelgono il valore massimo tra i figli, i nodi MIN scelgono il minimo. In questo modo la radice dell'albero acquisisce il valore della mossa ottimale che MAX dovrebbe eseguire.

È importante chiarire che Minimax non costruisce un percorso che porta MAX a vincere: **suggerisce soltanto la prossima mossa ottimale**, assumendo che l'avversario risponda nel modo migliore possibile. La vittoria non è garantita se la posizione di partenza è già sfavorevole.

Formula Minimax

$$\text{MiniMax}(s) = \begin{cases} \text{Utility}(s) & \text{se } s \text{ è terminale} \\ \max_a \text{MiniMax}(\text{Result}(s, a)) & \text{se Player}(s) = \text{MAX} \\ \min_a \text{MiniMax}(\text{Result}(s, a)) & \text{se Player}(s) = \text{MIN} \end{cases}$$

Complessità temporale: $O(b^m)$. Con alpha-beta pruning nel caso migliore: $O(b^{m/2})$.

Domanda: Cos'è l'Alpha-Beta Pruning? Come migliora Minimax?

L'alpha-beta pruning è un'ottimizzazione di Minimax che **elimina rami dell'albero di gioco che non possono influenzare la decisione finale**, senza cambiare il risultato. L'idea è mantenere due valori durante la visita: α (il miglior valore garantito finora per MAX) e β (il miglior valore garantito finora per MIN, ovvero il più basso).

La regola di potatura è la seguente: se durante l'esplorazione di un nodo MIN troviamo un valore $\leq \alpha$, possiamo smettere di espandere quel nodo (potatura beta), perché MAX non sceglierà mai questo ramo – ha già un'alternativa migliore. Simmetricamente, se durante un nodo MAX troviamo un valore $\geq \beta$, possiamo tagliare (potatura alfa), perché MIN non permetterà mai a MAX di arrivare qui.

Nel **caso migliore** – quando i nodi sono ordinati in modo ottimale, con le mosse migliori esaminate per prime – l'alpha-beta dimezza la profondità effettiva dell'albero: la complessità scende da $O(b^m)$ a $O(b^{m/2})$, il che equivale a poter cercare al doppio della profondità con la stessa quantità di calcolo. Nel **caso peggiore** (ordinamento pessimo) non dà nessun vantaggio.

È fondamentale capire che alpha-beta *non cambia la mossa scelta da Minimax*: produce esattamente lo stesso risultato, solo esplorando meno nodi. La potatura non influenza la correttezza perché i rami tagliati non avrebbero mai potuto essere scelti da un giocatore razionale.

6 Logica del Primo Ordine

Domanda: Cos'è la Forma Normale Congiuntiva (CNF)?

La Forma Normale Congiuntiva è una rappresentazione standard delle formule logiche come **congiunzione di clausole**, dove ogni clausola è una **disgiunzione di letterali**. Un letterale è un atomo o la sua negazione. Per esempio: $(A \vee \neg B) \wedge (B \vee C \vee \neg A)$ è in CNF.

La CNF è fondamentale perché è la forma richiesta dall'algoritmo di **risoluzione**. Qualsiasi formula della logica proposizionale o del primo ordine può essere convertita in CNF attraverso una procedura standard: si eliminano prima i bicondizionali ($\Leftrightarrow$)

e le implicazioni ($\Rightarrow$), poi si porta la negazione verso l'interno usando le leggi di De Morgan, e infine si distribuisce la disgiunzione sulla congiunzione.

Domanda: Cosa sono le clausole di Horn e perché sono importanti?

Una clausola di Horn è una clausola che contiene **al più un letterale positivo** (non negato). Per esempio, $\neg P(x) \vee Q(x) \vee \neg R(x)$ è di Horn perché ha un solo letterale positivo ($Q(x)$), mentre $P(x) \vee Q(x) \vee \neg R(x)$ *non* è di Horn perché ha due letterali positivi.

Le clausole di Horn sono importanti per due ragioni principali. Prima di tutto, l'inferenza su insiemi di clausole di Horn è decidibile in **tempo polinomiale**, mentre l'inferenza sulla logica del primo ordine in generale non è decidibile. Secondariamente, le clausole di Horn ammettono algoritmi di inferenza molto efficienti come il **forward chaining** e il **backward chaining**.

In forma implicativa, una clausola di Horn con un letterale positivo si scrive come $P_1 \wedge P_2 \wedge \dots \wedge P_k \Rightarrow Q$: le parti negative sono le premesse, la parte positiva è la conclusione. Questo corrisponde esattamente alla struttura delle regole nei sistemi esperti e in Prolog.

Domanda: Come funziona il Forward Chaining?

Il forward chaining è un algoritmo di inferenza **guidato dai dati**: parte dai fatti noti nella base di conoscenza e applica ripetutamente le regole le cui premesse sono soddisfatte, aggiungendo nuove conclusioni ai fatti noti. Continua fino al raggiungimento del goal o fino al punto fisso, ovvero fino a quando non si possono più derivare nuovi fatti.

Perché il forward chaining sia applicabile, è necessario che la base di conoscenza contenga *clausole di Horn* e che **tutti** i fatti nelle premesse di una regola siano già presenti in KB. Se anche uno solo dei fatti richiesti manca, la regola non può essere applicata.

Il **Modus Ponens Generalizzato** (MPG) è la versione del forward chaining per la logica del primo ordine: invece di lavorare su formule proposizionali, usa l'**unificazione** per trovare la sostituzione θ che rende le premesse della regola identiche ai fatti noti, e poi applica quella sostituzione alla conclusione.

Modus Ponens – formula

$$\frac{\alpha \quad \alpha \Rightarrow \beta}{\beta} \quad \text{MPG: } \frac{p_1, \dots, p_k \quad (P_1 \wedge \dots \wedge P_k \Rightarrow Q)\theta}{Q\theta}$$

dove θ è la sostituzione che unifica P_i con p_i per ogni i .

Domanda: Cos'è il Backward Chaining? Differenze con il Forward Chaining?

Il backward chaining è un algoritmo di inferenza **guidato dall'obiettivo**: parte dalla query che si vuole dimostrare e cerca *a ritroso* le regole la cui conclusione corrisponde alla query, tentando poi di dimostrare le premesse di quelle regole ricorsivamente. È la strategia alla base di **Prolog**.

La differenza fondamentale con il forward chaining è la direzione dell'inferenza. Il forward chaining parte dai fatti e deriva tutte le conseguenze possibili finché non si raggiunge il goal – può derivare molti fatti inutili lungo il percorso. Il backward chaining parte dal goal e determina solo quali fatti servono per dimostrarlo – è molto più efficiente quando la query è specifica e lo spazio degli obiettivi parziali è piccolo rispetto all'insieme di tutti i fatti derivabili.

Entrambi richiedono clausole di Horn per funzionare correttamente. La scelta tra i due dipende dal problema: il forward chaining è preferibile quando si hanno pochi fatti e molte query, mentre il backward chaining è preferibile con molti fatti e query specifiche.

Domanda: Cos'è l'unificazione? Cosa si intende per MGU?

L'unificazione è il processo di trovare una **sostituzione** θ – un'associazione di variabili a termini – tale che due espressioni diventino sintatticamente identiche dopo aver applicato θ . Per esempio, $P(x, f(y))$ e $P(a, f(b))$ si unificano con la sostituzione $\{x/a, y/b\}$: applicando θ , entrambe diventano $P(a, f(b))$.

Il **Most General Unifier** (MGU) è la sostituzione più generale tra tutte quelle che unificano due espressioni: non vincola le variabili più del necessario. Se esistono più sostituzioni unificanti, l'MGU è quella da cui si possono derivare tutte le altre. Per esempio, $\{x/y\}$ è più generale di $\{x/a, y/a\}$ perché la seconda si ottiene dalla prima con un'ulteriore sostituzione.

L'unificazione fallisce quando si incontra l'**occur check**: non si può unificare x con $f(x)$ perché si creerebbe una struttura infinita. In molte implementazioni pratiche (come Prolog) l'occur check viene omissso per motivi di efficienza, a rischio di risultati errati in casi particolari.

L'unificazione è il meccanismo fondamentale che rende possibile il Modus Ponens Generalizzato e la risoluzione in FOL: senza di essa non potremmo applicare regole universali a fatti specifici.

Domanda: Cos'è la risoluzione e come si usa per la refutazione?

La risoluzione è una regola di inferenza che, date due clausole contenenti un letterale e il suo complementare, produce una nuova clausola – il *risolvente* – eliminando quei letterali. Per esempio, da $(A \vee C)$ e $(\neg C \vee B)$ si ottiene il risolvente $(A \vee B)$: i letterali complementari C e $\neg C$ vengono eliminati.

In FOL, si usa prima l'**unificazione** per rendere complementari i letterali prima di risolvere. A differenza del forward chaining, la risoluzione non richiede clausole di Horn: funziona su qualsiasi clausola.

La **dimostrazione per refutazione** sfrutta questo principio: per dimostrare che

$KB \models \alpha$, si aggiunge $\neg\alpha$ alla base di conoscenza convertita in CNF, e si applicano ripetutamente le risoluzioni. Se si arriva alla **clausola vuota** $\square$ – ovvero a una contraddizione – significa che $KB \wedge \neg\alpha$ è insoddisfacibile, e quindi α è conseguenza logica di KB.

Domanda: Cosa sono i termini ground e le funzioni di Skolem?

Un termine è **ground** se non contiene variabili: contiene solo costanti o applicazioni di funzioni a costanti. Per esempio, Padre(Giovanni) e Anna sono termini ground, mentre Padre(x) non lo è perché contiene la variabile x . Una formula è ground se tutti i suoi termini lo sono.

Le **funzioni di Skolem** emergono durante la conversione di formule FOL in CNF, precisamente nella fase di *skolemizzazione*: l'eliminazione dei quantificatori esistenziali. Quando incontriamo $\exists x P(x)$ in una formula senza quantificatori universali esterni, sostituiamo x con una nuova costante di Skolem k . Quando invece il quantificatore esistenziale è dentro uno universale – per esempio $\forall y \exists x P(x, y)$ – sostituiamo x con una **funzione di Skolem** $f(y)$, perché il valore di x può dipendere da y . La skolemizzazione preserva la soddisfacibilità della formula, non la validità.

Domanda: Cosa sono tautologie e contraddizioni?

Una formula è una **tautologia** (o formula valida) se è vera in *tutti* i modelli possibili, indipendentemente da come si assegnano i valori di verità alle variabili. L'esempio classico è $A \vee \neg A$: qualunque valore abbia A , la disgiunzione è sempre vera. Le tautologie sono importanti perché il **teorema di deduzione** afferma che $KB \models \alpha$ se e solo se $(KB \Rightarrow \alpha)$ è una tautologia.

Una **contraddizione** (formula insoddisfacibile) è l'opposto: è falsa in tutti i modelli. $A \wedge \neg A$ è una contraddizione. Questo concetto è alla base della dimostrazione per refutazione: se $KB \wedge \neg\alpha$ è una contraddizione, allora $KB \models \alpha$.

7 Machine Learning

Domanda: Come si costruisce un albero di decisione?

La costruzione di un albero di decisione segue un approccio ricorsivo. Si parte dall'intero insieme di training e si verifica la condizione di terminazione: se tutte le istanze appartengono alla stessa classe, si crea una foglia con quella classe. Se il nodo contiene poche istanze (sotto una soglia di pre-pruning) si crea una foglia con la classe maggioritaria. Altrimenti, si sceglie l'attributo che produce lo split migliore.

La bontà di uno split si misura con il **guadagno di informazione**: si calcola l'entropia del nodo padre, poi la media pesata delle entropie dei nodi figli, e si sceglie l'attributo che massimizza la differenza. Si creano quindi tanti nodi figli

quanti sono i valori dell'attributo scelto, si partizionano le istanze tra di essi, e si applica la procedura ricorsivamente.

È essenziale distinguere il criterio di terminazione: il **numero di istanze sotto una soglia** è un valido criterio di pre-pruning. Non lo è invece l'overfitting del nodo (non rilevabile durante la costruzione) né un valore della funzione di valutazione (quella appartiene a Minimax, non agli alberi di decisione).

Domanda: Cosa misura l'entropia? E l'Information Gain?

L'**entropia** misura il grado di *impurezza* o disordine in un insieme di dati: ci dice quanto sono mescolate le classi in un nodo. La formula è:

$$H(t) = - \sum_i p(i|t) \log_2 p(i|t)$$

dove $p(i|t)$ è la proporzione di istanze della classe i nel nodo t . Quando tutte le istanze appartengono alla stessa classe, l'entropia è zero (nodo puro). Quando le classi sono distribuite in modo uniforme, l'entropia è massima.

L'**information gain** misura invece la *riduzione di entropia* prodotta da uno split sull'attributo A :

$$\text{Gain}(A) = H(\text{padre}) - \sum_v \frac{|D_v|}{|D|} H(D_v)$$

Si sceglie l'attributo con gain massimo perché è quello che riduce di più l'incertezza. Una cosa importante da ricordare è che né l'entropia né il guadagno di informazione si calcolano dalla matrice di confusione: appartengono alla fase di costruzione del modello sul training set, non alla fase di valutazione.

Domanda: Cosa si intende per overfitting? Cosa sono pre-pruning e post-pruning?

L'**overfitting** si verifica quando un modello si adatta troppo perfettamente al training set, imparando anche il rumore e le irregolarità casuali dei dati, invece di cogliere il pattern sottostante. Il risultato è un'alta accuratezza sul training ma basse prestazioni sui dati nuovi. Il principio di Occam suggerisce che, a parità di errore sul training, si preferisca il modello più semplice.

Il **pre-pruning** è una strategia per prevenire l'overfitting durante la costruzione dell'albero: si interrompe la ramificazione prima del completamento, quando il numero di istanze nel nodo scende sotto una soglia o il guadagno di informazione è troppo basso. Il rischio è l'underfitting: fermarsi troppo presto.

Il **post-pruning** è invece applicato dopo aver costruito l'albero completo: si esaminano i sottoalberi e si sostituiscono quelli che generalizzano male – verificato su un validation set – con semplici foglie etichettate con la classe maggioritaria. È in generale più affidabile del pre-pruning perché si dispone di una visione globale dell'albero. Il contesto del pruning è *esclusivamente* l'apprendimento automatico, non i CSP né i giochi.

Domanda: Cos'è il Perceptron? Quale problema lo ha reso famoso?

Il percettrone, proposto da Rosenblatt nel 1957, è il modello più semplice di neurone artificiale: riceve un vettore di input $\mathbf{x}$, calcola la combinazione lineare pesata $\mathbf{w} \cdot \mathbf{x} + b$ e applica una funzione di attivazione per produrre l'output. L'apprendimento avviene attraverso la **regola delta**: i pesi vengono aggiornati proporzionalmente all'errore commesso sull'istanza corrente:

$$w_j(t+1) = w_j(t) + \eta \cdot (d - o) \cdot x_j$$

Il percettrone implementa un *test lineare*: i pesi definiscono un iperpiano nello spazio degli input, e le istanze vengono classificate in base a quale lato dell'iperpiano si trovano.

Il problema che ha messo in luce i suoi limiti è il **problema dell'OR esclusivo (XOR)**. Minsky e Papert dimostrarono nel 1969 che il percettrone non può imparare la funzione XOR perché le sue quattro combinazioni di input non sono linearmente separabili: non esiste nessun iperpiano che separi correttamente i casi positivi da quelli negativi. Questo portò al primo “inverno dell'IA”, fino allo sviluppo del backpropagation per reti multistrato negli anni '80.

8 Matrice di Confusione

Domanda: Cos'è la matrice di confusione? Cosa si calcola da essa?

La matrice di confusione è uno strumento di valutazione dei modelli di classificazione: mette a confronto le etichette reali con quelle predette dal modello. Per una classificazione binaria, le quattro celle contengono i **veri positivi** (TP: predetto positivo, realmente positivo), i **falsi positivi** (FP: predetto positivo, realmente negativo), i **falsi negativi** (FN) e i **veri negativi** (TN).

Una prestazione ottimale corrisponde a tutti i valori concentrati sulla **diagonale principale** – ovvero TP e TN – con zero fuori dalla diagonale: significa che ogni istanza è stata classificata correttamente.

Le metriche che si calcolano dalla matrice di confusione includono l'**accuratezza** $= (TP + TN)/\text{totale}$, l'error rate $= 1 - \text{accuratezza}$, la precisione e il richiamo. Ciò che *non* si calcola dalla matrice è l'entropia, l'information gain o l'overfitting: queste grandezze appartengono alla costruzione del modello, non alla sua valutazione sulle predizioni.

9 Ontologie

Domanda: Cosa si intende per ontologia? Quali relazioni contiene?

Un'ontologia è una specifica formale di una concettualizzazione: definisce quali oggetti esistono in un dominio e come sono collegati tra loro. Le relazioni fondamentali sono quattro.

La relazione **isa** (is-a) esprime una **sottoclasse**: “virologo isa microbiologo” significa che virologo è una sottoclasse di microbiologo, e quindi ogni virologo è anche un microbiologo. La relazione **part-of** esprime la **composizione**: “capsomero part-of virus” indica che il capsomero è una componente strutturale del virus. La relazione **member** esprime l'**istanza**: “HIV member virus” indica che HIV è un esemplare specifico della classe virus, non una sottoclasse. Infine, un'espressione del tipo “member(X, C) $\Rightarrow$ P(X)” definisce una **proprietà di classe**: ogni membro di C ha la proprietà P.

Un insieme di sottocategorie che sono sia *disgiunte* che *esaustive* rispetto alla categoria padre costituisce una **partizione**. In una tassonomia, la proprietà corretta è che *le sottocategorie di ciascuna categoria* formino una partizione, non che l'intero insieme di tutte le categorie lo sia.

RDF, il Resource Description Framework alla base del Web Semantico, rappresenta la conoscenza come **triple** $\langle$ **soggetto, predicato, oggetto** $\rangle$ e non tramite clausole di Horn.

10 Frame Problem e Situation Calculus

Domanda: Cos'è il Frame Problem?

Il frame problem è una delle sfide più sottili nella rappresentazione della conoscenza per agenti che agiscono nel mondo: è l'incapacità di sapere quali proprietà di una situazione si **preservano** dopo l'applicazione di un'azione. Il problema non è descrivere cosa cambia – per quello bastano gli assiomi degli effetti – ma descrivere tutto ciò che *non* cambia senza dover elencare esplicitamente ogni singola proprietà invariata.

Per esempio, se un robot sposta un blocco, intuitivamente tutto il resto del mondo rimane uguale. Ma in una rappresentazione logica formale, come faccio a sapere che il colore del muro non è cambiato? Senza un meccanismo apposito, dovrei scrivere un assioma per ogni proprietà del mondo che non è stata modificata.

Nel **situation calculus** questo si risolve con gli **assiomi del successore di stato**: un fluente è vero nella situazione risultante dall'azione se e solo se l'azione lo ha reso vero, oppure era già vero e l'azione non lo ha reso falso. Questi assiomi catturano in modo compatto sia gli effetti sia la persistenza.

La definizione di un'azione nel situation calculus include le **precondizioni** (le proprietà che devono valere per applicare l'azione) e gli **effetti** (le proprietà che l'azione modifica). Le “proprietà atemporali” non fanno parte di questa definizione: sono assiomi della KB generale, non specifici di una singola azione.